Keras課堂實驗(五)

CIFAR-10 影像分類

高怡宣
陽明大學 生物醫學影像暨放射科學系

CIFAR-10 data set

影像辨識

The CIFAR-10 dataset (Canadian Institute For Advanced Research) is a collection of images that are commonly used for machine learning research. It contains 60,000 32x32 color images in 10 different classes, including: airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks. There are 6,000 images of each class. (from Wikipedia)

Data: Computer Science, University of Toronto, Canada
https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz

Code: Object Recognition with Convolutional Neural Networks in the Keras Deep Learning Library
by Jason Brownlee on July 1, 2016 in Deep Learning
https://machinelearningmastery.com/object-recognition-convolutional-neural-networks-keras-deep-learning-library/

python

ANACONDA

Keras: The Python Deep Learning library

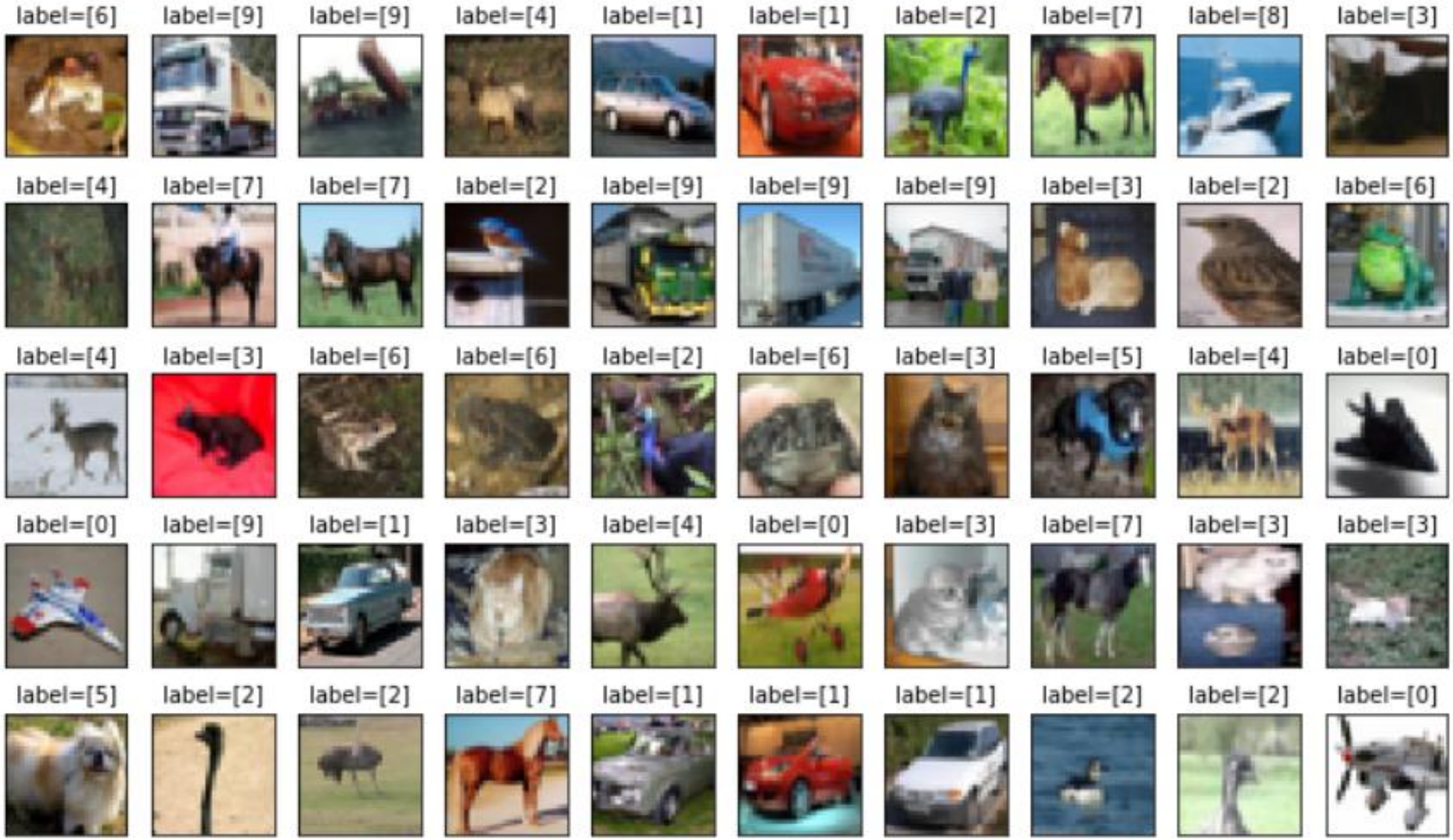
Keras

jupyter

TensorFlow

kaggle

0: airplanes, 1: cars, 2: birds, 3: cats, 4: deer, 5: dogs, 6: frogs, 7: horses, 8: ships, 9: trucks



雲端硬碟
搜尋雲端硬碟
新增
我的雲端硬碟
電腦
與我共用
近期存取
已加星號
我的雲端硬碟 > CIFAR
名稱
擁有者
上次修改日期
Keras_Cifar10_CNN.ipynb
我
上午8:22

Keras_Cifar10_CNN.ipynb

使用「Colaboratory」開啟

{"nbformat":4,"nbformat_minor":0,"metadata":{"colab":
{"name":"Keras_Cifar10_CNN.ipynb","version":"0.3.2","provenance":[],"collapsed_sections":
[]},"kernelspec":{"name":"python3","display_name":"Python
3"},"accelerator":"GPU"},"cells":[{"metadata":
{"colab_type":"text","id":"OqCZ0wByIrmf"},"cell_type":"markdown","source":["# CIFAR-10
image analysis using CNN\n","### Object Recognition with Convolutional Neural Networks in
the Keras Deep Learning Library, by Jason Brownlee on July 1, 2016 in Deep
Learning\n","##### code modified from: https://machinelearningmastery.com/object-
recognition-convolutional-neural-networks-keras-deep-learning-library/\n","##### code
modified from: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴"]},{"metadata":
{"colab_type":"text","id":"_16rVccJNdtV"},"cell_type":"markdown","source":["# 1. Import
Library"]},{"metadata":{"colab_type":"code","id":"yNv36SvWNdtX","colab":
{}},"cell_type":"code","source":["# Simple CNN model for CIFAR-10\n","# import
numpy\n","\n","from keras.datasets import cifar10\n","from keras.utils import
np_utils\n","import matplotlib.pyplot as plt\n","import pandas as pd\n","import numpy as
np\n","\n","np.random.seed(3)"],"execution_count":0,"outputs":[]},{"metadata":
{"colab_type":"text","id":"oYKaEFUaNdti"},"cell_type":"markdown","source":["# 資料準備"]},
{"metadata":{"colab_type":"code","id":"cpOJDhvoNdtj","outputId":"0910807d-cc40-4ef4-d09c-
b7fa20234f55","colab":{"base_uri":"https://localhost:8080/","height":52},"executionInfo":
{"status":"ok","timestamp":1543278572500,"user_tz":-480,"elapsed":45770,"user":
{"displayName":"高怡宣老師","photoUrl":"https://lh5.googleusercontent.com/-
U9J4qMdQuYo/AAAAAAAAAAI/AAAAAAAAADc/D3qNuqyRU0g/s64/photo.jpg","userId":"115482693222803 43
912"}}},"cell_type":"code","source":["(x_train_image,y_train_label),

Keras_Cifar10_CNN.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单元格 单元格

CIFAR-10 image analysis using CNN

Object Recognition with Convolutional Neural Networks in the Keras Deep Learning Library, by Jason Brownlee Learning

code modified from: https://machinelearningmastery.com/object-recognition-convolutional-neural-networks-keras-deep-learning-library/

code modified from: TensorFlow+Keras[深度學習]人工智慧實務應用 / 林大貴

1. Import Library

```
[ ]  # Simple CNN model for CIFAR-10
     # import numpy

     from keras.datasets import cifar10
     from keras.utils import np_utils
     import matplotlib.pyplot as plt
```

Keras_Cifar10_CNN.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单

全部运行	Ctrl+F9
运行当前单元格之前的所有单元格	Ctrl+F8
运行光标所在的单元格	Ctrl+Enter
运行所选单元格的内容	Ctrl+Shift+Enter
运行当前单元格以及其后的所有单元格	Ctrl+F10
中断执行	Ctrl+M I
重新启动代码执行程序...	Ctrl+M .
重新启动并运行所有单元格...	
重置所有代码执行程序...	
更改运行时类型	
管理会话	

CIFAR-10 ima

Object Recognition with **Learning Library, by Jason Brownlee**
Learning

code modified from: https://mach **orks-keras-deep-learning-library/**

code modified from: TensorFlow+

1. Import Libr

```
# Simple CNN model for CIFAR-10
# import numpy

from keras.datasets import cifar10
from keras.utils import np_utils
import matplotlib.pyplot as plt
import pandas as pd
```

Keras_Cifar10_CNN.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单元格 单元格 已连

CIFAR-10 image analysis usi

Object Recognition with Convolutional Neural Ne ... rownlee on July 1, 2016 in Deep Learning

code modified from: https://machinelearningmastery.com/object-re

code modified from: TensorFlow+Keras[深度學習]人工智慧實務應用

1. Import Library

```
# Simple CNN model for CIFAR-10
# import numpy

from keras.datasets import cifar10
from keras.utils import np_utils
import matplotlib.pyplot as plt
import pandas as pd
```

笔记本设置

运行时类型

Python 3

硬件加速器

GPU

None

GPU

TPU

保存此笔记本时忽 ... 格输出项

取消 保存

Keras_Cifar10_CNN.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

代码 文字 单

全部运行	Ctrl+F9
运行当前单元格之前的所有单元格	Ctrl+F8
运行光标所在的单元格	Ctrl+Enter
运行所选单元格的内容	Ctrl+Shift+Enter
运行当前单元格以及其后的所有单元格	Ctrl+F10
中断执行	Ctrl+M I
重新启动代码执行程序...	Ctrl+M .
重新启动并运行所有单元格...	
重置所有代码执行程序...	
更改运行时类型	
管理会话	

CIFAR-10 ima

Object Recognition with **Learning Library, by Jason Brownlee**

Learning

code modified from: https://mach **orks-keras-deep-learning-library/**

code modified from: TensorFlow+

1. Import Libr

```
# Simple CNN model for CIFAR-10
# import numpy

from keras.datasets import cifar10
from keras.utils import np_utils
import matplotlib.pyplot as plt
import pandas as pd
```

1. Import Library

```
# Simple CNN model for CIFAR-10
# import numpy

from keras.datasets import cifar10
from keras.utils import np_utils
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

np.random.seed(3)
```

資料準備

```
(x_train_image, y_train_label), (x_test_image, y_test_label)=cifar10.load_data()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
 49111040/170498071 [=======>.....................] - ETA: 27s
```

1. Import Library

In [1]:
```
# Simple CNN model for CIFAR-10
# import numpy

from keras.datasets import cifar10
from keras.utils import np_utils
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

np.random.seed(3)
```

```
Using TensorFlow backend.
```

資料準備

```
(x_train_image,y_train_label),(x_test_image,y_test_label)=cifar10.load_data()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170500096/170498071 [==============================] - 37s 0us/step
```

```
print("train data:",'images:',x_train_image.shape, " labels:",y_train_label.shape)
print("test  data:",'images:',x_test_image.shape, " labels:",y_test_label.shape)
```

```
train data: images: (50000, 32, 32, 3)  labels: (50000, 1)
test  data: images: (10000, 32, 32, 3)  labels: (10000, 1)
```

View the images

In [4]:
```
fig = plt.gcf()
fig.set_size_inches(12,14)

for i in range(0,10):
    ax=plt.subplot(5,5,1+i)
    ax.imshow(x_train_image[i], cmap='binary')
    title= "label=" +str(y_train_label[i])
    ax.set_title(title,fontsize=10)
    ax.set_xticks([]);ax.set_yticks([])
plt.show()
```

0: airplanes, 1: cars, 2: birds, 3: cats, 4: deers, 5: dogs, 6: frogs, 7: horses, 8: ships, 9: trucks

Prepare the data

```
x_train_image_normalize = x_train_image.astype('float32') / 255.0
x_test_image_normalize = x_test_image.astype('float32') / 255.0
```

```
from keras.utils import np_utils
y_train_label_OneHot = np_utils.to_categorical(y_train_label)
y_test_label_OneHot = np_utils.to_categorical(y_test_label)
```

```
y_test_label_OneHot.shape
```

```
(10000, 10)
```

建立模型

```
from keras.models import Sequential
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Conv2D, MaxPooling2D, ZeroPadding2D
from keras.constraints import maxnorm
from keras.optimizers import SGD
num_classes = y_test_label_OneHot.shape[1]
```

```
num_classes = y_test_label_OneHot.shape[1]
```

```
# Create the model
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(32, 32, 3), padding=
model.add(Dropout(0.2))
model.add(Conv2D(32, (3, 3), activation='relu', padding='same'
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(512, activation='relu', kernel_constraint=maxn
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
```

```
model.compile(loss='categorical_crossentropy',
              optimizer='adam', metrics=['accuracy'])
```

載入之前訓練的模型

```
# load HDF5 file
from keras.models import load_model

try:
    model = load_model('SaveModel/my_cifarCnnModel.h5')
    print("載入模型成功!繼續訓練模型")
except :
    print("載入模型失敗!開始訓練一個新模型")
```

載入模型失敗!開始訓練一個新模型

訓練模型

```
print(model.summary())
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 32, 32, 32)        896
_________________________________________________________________
dropout_1 (Dropout)          (None, 32, 32, 32)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 32, 32, 32)        9248
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 16, 16, 32)        0
```

```
_________________________________________________________________
flatten_1 (Flatten)          (None, 8192)              0
_________________________________________________________________
dense_1 (Dense)              (None, 512)               4194816
_________________________________________________________________
dropout_2 (Dropout)          (None, 512)               0
_________________________________________________________________
dense_2 (Dense)              (None, 10)                5130
=================================================================
Total params: 4,210,090
Trainable params: 4,210,090
Non-trainable params: 0
_________________________________________________________________
None
```

```
# saves the model weights after each epoch if the validation loss decreased
from keras.callbacks import ModelCheckpoint
checkpointer = ModelCheckpoint(filepath='weights.hdf5',
                               verbose=1, save_best_only=True)
```

```
train_history=model.fit(x_train_image_normalize, y_train_label_OneHot,
                        validation_split=0.2,
                        epochs=40, batch_size=128, verbose=1,
                        callbacks=[checkpointer])
```

```
train_history=model.fit(x_train_image_normalize, y_train_label_OneHot,
                        validation_split=0.2,
                        epochs=40, batch_size=128, verbose=1,
                        callbacks=[checkpointer])
```

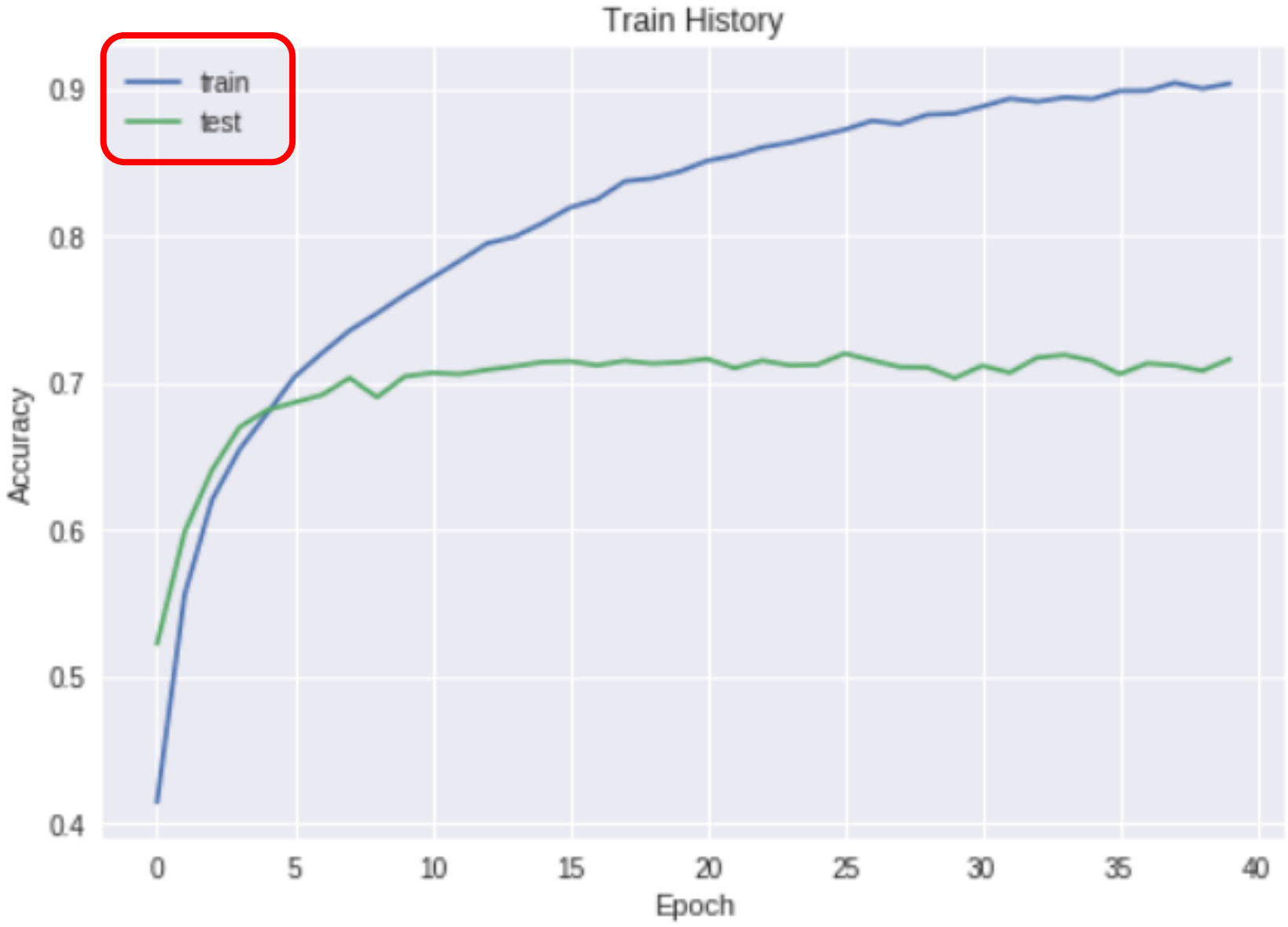
```
Train on 40000 samples, validate on 10000 samples
Epoch 1/40
40000/40000 [==============================] - 14s 343us/step - loss: 1.6111 -
acc: 0.4146 - val_loss: 1.3860 - val_acc: 0.5223

Epoch 00001: val_loss improved from inf to 1.38601, saving model to weights.hd
f5
```

```
Epoch 2/40
40000/40000 [==============================] - 9s 225us/step - loss: 1.2542 -
acc: 0.5556 - val_loss: 1.1658 - val_acc: 0.5983

Epoch 00002: val_loss improved from 1.38601 to 1.16584, saving model to weight
s.hdf5
Epoch 3/40
40000/40000 [==============================] - 9s 225us/step - loss: 1.0760 -
acc: 0.6203 - val_loss: 1.0338 - val_acc: 0.6406

Epoch 00003: val_loss improved from 1.16584 to 1.03377, saving model to weight
s.hdf5
Epoch 4/40
40000/40000 [==============================] - 9s 226us/step - loss: 0.9771 -
acc: 0.6546 - val_loss: 0.9470 - val_acc: 0.6698

Epoch 00004: val_loss improved from 1.03377 to 0.94702, saving model to weight
s.hdf5
```

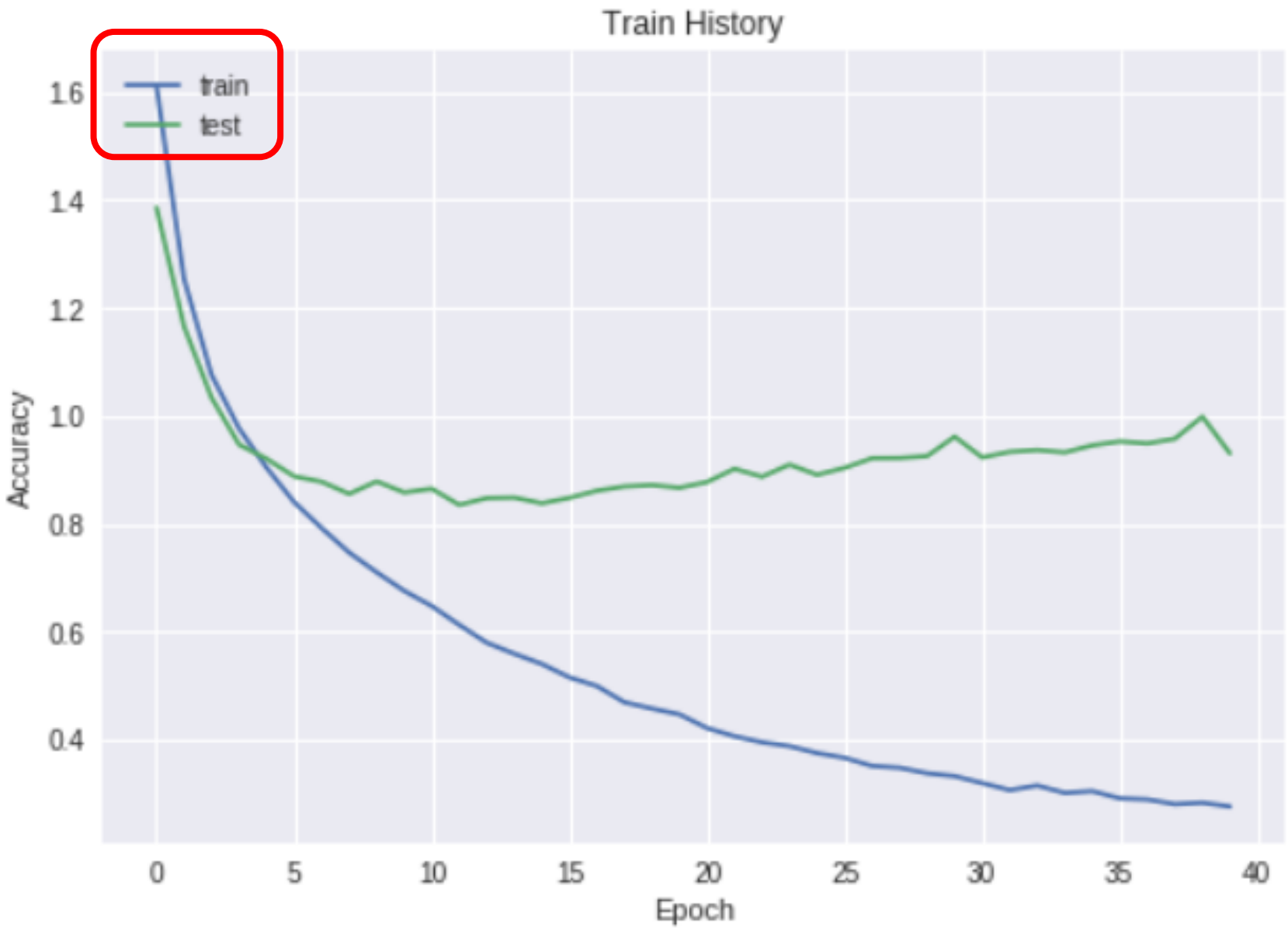
```
Epoch 39/40
40000/40000 [==============================] - 9s 224us/step - loss: 0.2837 -
acc: 0.9001 - val_loss: 0.9987 - val_acc: 0.7081

Epoch 00039: val_loss did not improve from 0.83526
Epoch 40/40
40000/40000 [==============================] - 9s 225us/step - loss: 0.2768 -
acc: 0.9036 - val_loss: 0.9308 - val_acc: 0.7161

Epoch 00040: val_loss did not improve from 0.83526
```

In [0]:
```
import matplotlib.pyplot as plt
def show_train_history(train_acc,test_acc):
    plt.plot(train_history.history[train_acc])
    plt.plot(train_history.history[test_acc])
    plt.title('Train History')
    plt.ylabel('Accuracy')
    plt.xlabel('Epoch')
    plt.legend(['train', 'test'], loc='upper left')
    plt.show()
```

```
In [16]: show_train_history('acc','val_acc')
```



```
In [17]: show_train_history('loss','val_loss')
```



評估模型準確率

In [18]:
```
scores = model.evaluate(x_test_image_normalize,
                        y_test_label_OneHot, verbose=0)
scores[1]
```

Out[18]: 0.7144

進行預測

```
In [0]: prediction=model.predict_classes(x_test_image_normalize)
```

```
In [20]: prediction[:10]
Out[20]: array([3, 8, 8, 0, 6, 6, 5, 6, 3, 1])
```

查看預測結果

```
label_dict={0:"airplane",1:"automobile",2:"bird",3:"cat",4:"deer",
            5:"dog",6:"frog",7:"horse",8:"ship",9:"truck"}
```

```
def plot_images_labels_prediction(images,labels,prediction,
                                  idx,num=10):
    fig = plt.gcf()
    fig.set_size_inches(12, 14)
    if num>25: num=25
    for i in range(0, num):
        ax=plt.subplot(5,5, 1+i)
        ax.imshow(images[idx], cmap='binary')
        title= "label=" +str(labels[idx])
        if len(prediction)>0:
            title+=",predict="+str(prediction[idx])

        ax.set_title(title,fontsize=10)
        ax.set_xticks([]);ax.set_yticks([])
        idx+=1
    plt.show()
```

0: airplanes, 1: cars, 2: birds, 3: cats, 4: deer, 5: dogs, 6: frogs, 7: horses, 8: ships, 9: trucks


```
plot_images_labels_prediction(x_test_image,y_test_label, prediction,0,10)
```

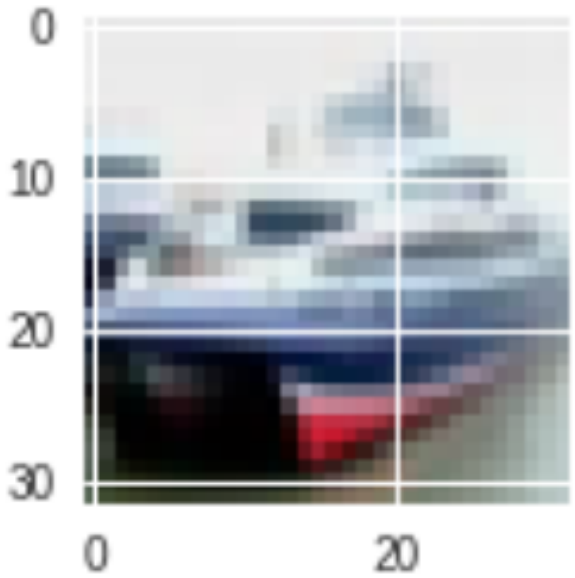
查看預測機率

```
Predicted_Probability=model.predict(x_test_image_normalize)
```

```
def show_Predicted_Probability(y,prediction,
                               x_img,Predicted_Probability,i):
    print('label:',label_dict[y[i][0]],
          'predict:',label_dict[prediction[i]])
    plt.figure(figsize=(2,2))
    plt.imshow(np.reshape(x_test_image[i],(32, 32,3)))
    plt.show()
    for j in range(10):
        print(label_dict[j]+
              ' Probability:%1.9f'%(Predicted_Probability[i][j]))
```

```
show_Predicted_Probability(y_test_label,prediction,
                           x_test_image,Predicted_Probability,1)
```

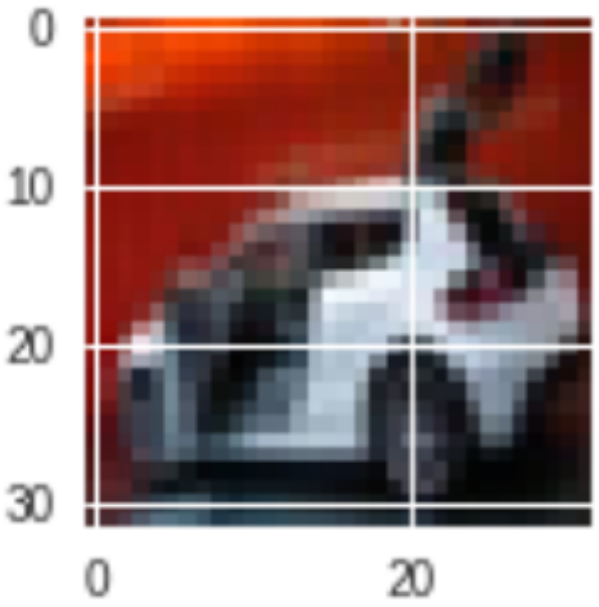
```
label: ship predict: ship
```




```
airplane Probability:0.000120721
automobile Probability:0.052998047
bird Probability:0.000000000
cat Probability:0.000000000
deer Probability:0.000000000
dog Probability:0.000000000
frog Probability:0.000000000
horse Probability:0.000000000
ship Probability:0.946880996
truck Probability:0.000000177
```

```
show_Predicted_Probability(y_test_label,prediction,
                           x_test_image,Predicted_Probability,6)
```

```
label: automobile predict: dog
```




```
airplane Probability:0.000364993
automobile Probability:0.277741164
bird Probability:0.000000294
cat Probability:0.038668558
deer Probability:0.000001007
dog Probability:0.682387710
frog Probability:0.000127588
horse Probability:0.000172766
ship Probability:0.000011233
truck Probability:0.000524789
```

confusion matrix

In [28]:
```
import pandas as pd
print(label_dict)
pd.crosstab(y_test_label.reshape(-1),prediction,
            rownames=['label'],colnames=['predict'])
```

predict	0	1	2	3	4	5	6	7	8	9
label										
0	753	21	53	21	22	9	7	16	55	43
1	13	825	7	9	5	5	8	2	25	101
2	63	4	554	47	116	80	69	42	15	10
3	18	8	63	505	86	180	73	37	11	19
4	12	2	50	36	767	27	46	48	10	2
5	8	4	47	163	69	630	19	44	8	8
6	8	5	52	44	67	44	767	3	4	6
7	7	8	30	35	80	57	8	755	2	18
8	72	46	11	14	10	9	5	9	778	46
9	30	73	6	14	13	10	5	13	26	810

Save model to JSON

In [0]:
```
model_json = model.to_json()
with open("cifarCnnModel.json", "w") as json_file:
    json_file.write(model_json)
```

Save Model to YAML

In [0]:
```
model_yaml = model.to_yaml()
with open("cifarCnnModel.yaml", "w") as yaml_file:
    yaml_file.write(model_yaml)
```

Save Weight to h5

In [32]:
```
model.save_weights("cifarCnnModel.h5")
print("Saved model to disk")
```

```
Saved model to disk
```

In [35]:
```
! ls -l
```

```
total 65856
-rw-r--r-- 1 root root 16862488 Nov 27 00:12 cifarCnnModel.h5
-rw-r--r-- 1 root root     3026 Nov 27 00:12 cifarCnnModel.json
-rw-r--r-- 1 root root     3241 Nov 27 00:12 cifarCnnModel.yaml
drwxr-xr-x 2 root root     4096 Nov 21 17:36 sample_data
-rw-r--r-- 1 root root 50558144 Nov 27 00:09 weights.hdf5
```

In [0]:
```
from google.colab import files
files.download('cifarCnnModel.h5')
```

In [0]:
```
files.download('cifarCnnModel.yaml')
```

In [0]:
```
files.download('cifarCnnModel.json')
```

Other methods to save model and parameters

```
# save model to HDF5 file
model.save('my_model.h5')
del model  # deletes the existing model

from keras.models import load_model
model = load_model('my_model.h5')
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 32, 32, 32)        896
```

```
# save and load weights
model.save_weights('my_model_weights.h5')
model.load_weights('my_model_weights.h5')
```

```
# save and load network using json
from keras.models import model_from_json
json_string = model.to_json()
del model  # deletes the existing model

model = model_from_json(json_string)
model.compile(loss='categorical_crossentropy', optimizer='Adam'
model.load_weights('my_model_weights.h5')
model.summary()
```

Keras_Cifar10_CNN.ipynb

文件 修改 视图 插入 代码执行程序 工具 帮助

打开云端硬盘中所在位置

在 Playground 模式下打开

新建 Python 2 记事本

新建 Python 3 记事本

打开笔记本... Ctrl+O

上传笔记本...

重命名...

移至回收站

在云端硬盘中保存一份副本...

将副本另存为 GitHub Gist...

在 GitHub 中保存一份副本...

保存 Ctrl+S

保存并标记修订版本 Ctrl+M S

修订历史记录

下载 .ipynb

下载 .py

```
[4
2)                    0
)                     5130
==========================
```

Homework:

(1) Find a Keras program from the web that can increase the accuracy to 85% for the CIFAR-10 data set.

(2) Implement the program.

Thank you for your attention